

# ResumeIQ -- Demo Video Script

Target length: 5-6 minutes · Format: screen recording of the live app (resumeiq-s3xc.onrender.com) with voiceover, or narrated live if presenting synchronously.

This is a recording script, not a video file -- producing the actual narrated recording needs a human (or screen-recording + voice tooling). Every number and quote below was pulled from a real run against the fixture recommended below, not written from memory, so you can record straight from it.

## Before recording

- 1 Open the live app 1-2 minutes before you start recording, so the free-tier cold start (~30-50s) doesn't happen on camera.
- 2 Have a resume file ready to upload. **tests/fixtures/strong\_resume.docx** works well -- mixed strong/weak bullets, with and without metrics, so every section has something worth pointing at.
- 3 Pick **"Web Developer"** as the target role for that fixture. Verified scores for this exact combination:
  - Resume Score **89** (~97th percentile)
  - ATS Score **42** (~66th percentile) -- a partial match, more convincing on camera than a trivial 0 or 100
  - Impact Score **78** (~97th percentile)
  - Role Fit: **Backend Developer scores 63** against the same resume -- 21 points higher than the selected Web Developer (42), so the honest "this resume actually fits X better" callout will fire on camera without you having to force it.
- 4 If you want the dark/particle background visible from the very first frame, load the page in dark mode (the moon icon in the header) before recording -- it reads better on screen than light mode.

## Script

### [0:00-0:20] — Hook + problem

"Most resumes get filtered by an ATS before a human ever reads them, and most resume-review tools either give vague feedback or hide behind a black-box AI that can't explain its scoring. ResumeIQ is different -- everything you're about to see traces back to a specific, inspectable rule. No LLM calls, no embeddings, and the interface says so up front."

### [0:20-0:55] — Upload flow + the role picker

"Here's the landing page." (*Show the homepage -- let the particle background and the neon button register for a second.*)

"First, I'll pick a target role." (*Click the role field -- it opens into a listbox with an icon and a one-line description for each of the 8 roles, not a plain dropdown.*)

"This isn't just a visual upgrade -- it's fully keyboard-operable: arrow keys, Enter, Escape, even typing a letter jumps to a matching role. I'll pick Web Developer." (*Select it, then drag the resume file in.*) "PDF or DOCX, up to 5MB." (*Click Analyze.*)

### [0:55-1:30] — Dashboard overview

"This is the report -- three headline scores at the top, each with a percentile underneath, plus a chart comparing all three at a glance." *(Scroll to show the tiles, the percentiles, and the bar chart.)*

"That percentile is worth pausing on: it's not compared against other ResumeIQ users -- this app doesn't have that data, and I'm not going to pretend it does. It's a documented statistical model of a typical early-career resume, and the footnote right here says exactly that. There's a whole FAQ entry about it if you want the details."

### [1:30-2:05] — Resume Score breakdown

"The Resume Score breaks into six categories straight from the brief -- structure, skills, education, projects, contact info, completeness." *(Scroll to the breakdown bars.)*

"Section detection is header-based -- it looks for short lines matching known section keywords, so a bullet that just mentions 'skills' mid-sentence doesn't accidentally register as a Skills section."

### [2:05-2:45] — ATS keyword checker + fuzzy matching + inline evidence

"The ATS checker compares the resume against a role-specific keyword bank." *(Scroll to the ATS section, point at matched/missing badges.)*

"This is one of the differentiators -- not just exact string matching. That 'SYNONYM' tag means the resume said one thing and the keyword bank listed the equivalent term; elsewhere it catches typos through fuzzy string matching. I calibrated that fuzzy threshold empirically, not by guessing -- a looser one would've credited a resume that only mentions Cython with knowing Python, since they score identically similar to a real typo. That's written up as a documented limitation, not something I found by accident."

"And scroll down to the extracted text preview" *(scroll to the bottom of the report)* "-- every matched keyword is highlighted right in your own resume text, color-coded by how it matched. Nothing here is a claim you have to take on faith."

### [2:45-3:15] — Role Fit Across Roles

"One more thing this report does: it doesn't just score you against the role you picked." *(Scroll to Role Fit Across Roles.)*

"It runs the same keyword check against all 8 roles, and when another role fits meaningfully better, it says so. For this resume, Web Developer scored 42 -- but Backend Developer scores 63. The dashboard flags that directly instead of leaving you to guess whether you picked the right target."

### [3:15-3:50] — Impact Score differentiator

"The other differentiator is the Impact Score." *(Scroll to the bullet list.)*

"Every bullet gets analyzed for three things -- strong or weak opening verb, a quantified result, and passive voice -- all through spaCy's dependency parser, not a model call." *(Point at a flagged bullet and a strong one.)*

"'Responsible for maintaining CI/CD pipelines' -- flagged weak, no metric. Compare that to 'Led a team of 4 engineers... increasing conversion by 18%' -- strong verb, metric detected."

### [3:50-4:15] — Smart feedback + PDF export

"All of that gets synthesized into this feedback list -- prioritized, specific suggestions, not generic advice." *(Scroll to Smart Feedback, read one or two aloud.)*

"And if you want this as a file instead of a webpage" *(click Download PDF Report)* "-- that's a real PDF generated on the server, not just a browser printout. The Print option is still there too, for a literal copy of the page."

### [4:15-4:45] — Wrap-up

"Everything here -- the scoring rubric, the ATS matching, the impact analysis, even the percentile -- is rule-based and documented, including where it deliberately falls short, like passive-voice detection only catching canonical 'was/were' constructions. There's an FAQ on the homepage that answers the questions people actually ask, like whether any of this touches an LLM." *(Switch to a new tab and open the GitHub repo directly -- the footer no longer links out, by design, so don't go looking for it there.)*

"The full source, 119 tests, and a project report walking through the architecture and the bugs I caught along the way are all in the repo. Thanks for watching."

## Optional B-roll / cutaways (if editing)

- The role picker's keyboard navigation (arrow keys moving the highlight, Enter selecting).
- The particle background and neon button hover state, in dark mode.
- The FAQ accordion opening/closing on the homepage.
- The downloaded PDF report opened side-by-side with the live dashboard.
- The mobile-responsive layout (resize the browser or use dev tools device mode).
- A quick look at the test suite passing (**pytest tests/ -v**).